CHAPTER 1

Transformer Architectures and Decoding Strategies

Chapter **??** established the foundational mathematics of the canonical Transformer, detailing how multi-head self-attention, cross-attention, and position-wise feed-forward networks overcome the sequential bottlenecks of recurrent architectures. While that framework was originally formulated as a dual-stack encoder-decoder for bilingual machine translation, subsequent deep learning research recognized that the encoder and decoder perform distinct representational functions that can be decoupled and specialized for different NLP tasks.

This structural divergence produced three principal architectural paradigms: unified encoder-decoder models conditioned on arbitrary task prefixes, encoder-only models leveraging unconstrained bidirectional context for sequence comprehension, and decoder-only models employing causal autoregression for open-ended generative synthesis. Concurrently, generating coherent natural text from autoregressive distributions revealed fundamental limitations in classical maximization decoding algorithms, necessitating the development of probabilistic truncation heuristics.

This chapter provides an exhaustive theoretical and empirical examination of Transformer architectural specializations and decoding mechanics. We begin by analyzing the multi-task text-to-text paradigm embodied by T5, dissecting its prefix-conditioning mechanism and dual-stack computational bottlenecks. We then deconstruct encoder-only representations through BERT, formalizing its tripartite input embedding structure, self-supervised pre-training objectives, emergent attention head specializations, and downstream task adaptation heads. Next, we formalize the decoder-only causal architecture, deriving its exact autoregressive probability factorization, causal masking mechanics, and computational rationale for modern large language models. Finally, we dissect the decoding spectrum, formulating deterministic and stochastic sampling strategies and evaluating their failure modes and empirical behaviors through rigorous mathematical formulations and controlled generation benchmarks.

1.1 Encoder-Decoder Models

The canonical Transformer proposed by Vaswani et al. [**?**] was engineered specifically for sequence-to-sequence machine translation, mapping an input sequence in a source language to a corresponding sequence in a target language. In that original paradigm, model weights were strictly specialized for an isolated task and language pair. The text-to-text transfer framework introduced by Raffel et al. [**?**] fundamentally reimagined this dual-stack topology, proving that a single shared parameterization could execute heterogeneous natural language processing tasks by unifying their input-output formats into a shared string-to-string interface.

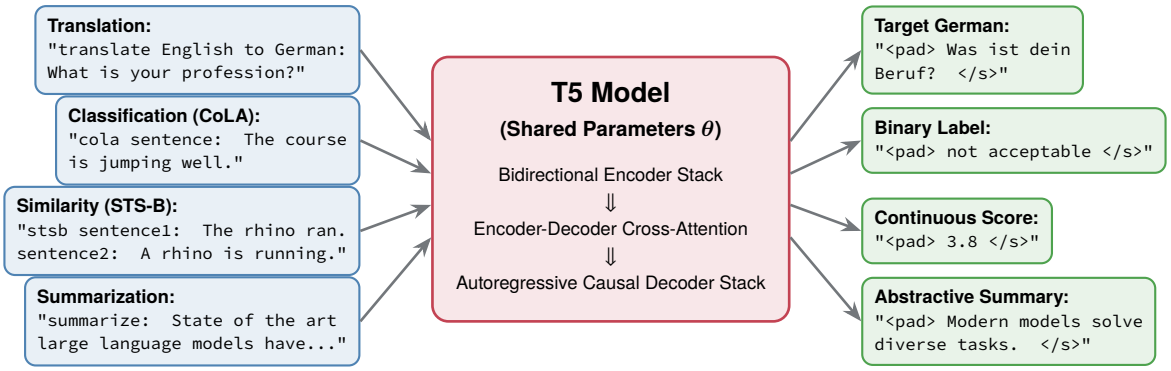


Figure 1.1: The T5 unified text-to-text framework [**?**]. Heterogeneous downstream tasks are cast into a single uniform string-to-string format conditioned by explicit task prefixes, utilizing identical loss functions and decoding routines.

1.1.1 The unified text-to-text framework and T5

Prior to the introduction of unified text-to-text architectures, empirical natural language processing relied on fragmented task-specific design conventions. Researchers typically affixed specialized classification layers atop an encoder for sentiment analysis, token-level linear classifiers for named entity recognition, regression heads for semantic textual similarity, or specialized sequence-to-sequence decoders for summarization and translation. Each downstream objective demanded a bespoke loss formulation, architectural head modification, and dedicated optimization schedule.

The Text-to-Text Transfer Transformer (T5) introduced by Raffel et al. [**?**] resolved this fragmentation by reframing every natural language objective as a text generation problem. Formally, let $\mathcal{V}^*$ denote the set of all arbitrary-length token sequences formed from vocabulary $\mathcal{V}$. Regardless of whether the underlying target is continuous regression, categorical classification, span extraction, or sequence translation, both the model input $\mathbf{x} \in \mathcal{V}^*$ and output $\mathbf{y} \in \mathcal{V}^*$ are represented exclusively as plain text strings:

$$f_{\boldsymbol{\theta}} : \mathcal{V}^* \to \mathcal{V}^*, \quad \mathbf{y} = f_{\boldsymbol{\theta}}(\mathbf{x}). \tag{1.1}$$

To enable a single invariant set of network parameters $\boldsymbol{\theta}$ to distinguish between competing functional objectives, T5 introduces *task-specific conditioning prefixes*. The user or training pipeline prepends a discrete textual indicator to the source sequence specifying the operational transformation desired. Figure 1.1 illustrates this unified conditioning paradigm across four disparate NLP tasks.

The mathematical elegance of this formulation lies in the universal applicability of the standard autoregressive maximum-likelihood objective. For any dataset $\mathcal{D} = \{(\mathbf{x}^{(m)}, \mathbf{y}^{(m)})\}_{m=1}^{M}$, the network is trained using token-level teacher-forced cross-entropy:

$$\mathcal{L}_{\text{seq2seq}}(\boldsymbol{\theta}) = -\sum_{m=1}^{M} \sum_{t=1}^{T_m} \log P\left(y_t^{(m)} \mid y_{<t}^{(m)}, \mathbf{x}^{(m)}; \boldsymbol{\theta}\right), \tag{1.2}$$

where $y_t^{(m)}$ represents the t-th target token of sequence m and T_m is its length. Continuous regression tasks such as STS-B are handled by requiring the model to emit decimal digit tokens (e.g., `"3.8"`), which are parsed back to floating-point values at inference. Binary classification tasks emit string literals (`"acceptable"` or `"not acceptable"`).

Crucially, as emphasized in course lectures, this prefix-conditioning paradigm must not be confused with modern instruction tuning. In T5, the model does not possess open-ended natural language intent comprehension. The prefix strings are rigid, memorized syntactic conditioning

triggers encountered during multi-task pre-training. If an unobserved or perturbed phrasing is supplied (for example, substituting `"can you translate from English to German:"` in place of `"translate English to German:"`), the conditioning signal degrades significantly because the underlying weights merely map memorized prompt templates to target output manifolds.

1.1.2 Architectural modifications and pre-training in T5

Beyond its operational text-to-text unification, T5 introduced key architectural deviations from the original 2017 Transformer specification to improve optimization stability and representation quality.

First, T5 dispenses with fixed absolute sinusoidal positional encodings $\mathbf{p}_i \in \mathbb{R}^{d_{\text{model}}}$. Instead, it adopts a parameterized *relative position bias*. Under this formulation, the scalar pre-softmax attention score between query token i and key token j is augmented by a learnable scalar offset $b_{i,j}$:

$$A_{i,j} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} + b_{i,j}, \tag{1.3}$$

where $b_{i,j}$ depends strictly on the relative displacement $(i - j)$. To bound the parameter footprint while retaining long-range sensitivity, displacements are mapped into discrete logarithmic buckets. Small relative distances ($|i - j| < 8$) receive exact dedicated bias parameters, whereas larger offsets are compressed logarithmically up to a maximum context window. This structure enforces translation invariance along sequence positions and enables extrapolation across varying sequence lengths.

Second, T5 simplifies the normalization topology by utilizing a modified root-mean-square normalization where the activation vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ is normalized strictly by its second moment without subtracting the mean:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d_{\text{model}}} \sum_{k=1}^{d_{\text{model}}} x_k^2 + \epsilon}} \odot \boldsymbol{\gamma}, \tag{1.4}$$

where $\boldsymbol{\gamma} \in \mathbb{R}^{d_{\text{model}}}$ is a learned scaling vector and $\epsilon > 0$ guarantees numerical stability. Normalization is placed on the input paths of each sub-block (*Pre-LN*), stabilizing backpropagation gradients through residual connections.

Third, for self-supervised pre-training over the Colossal Clean Crawled Corpus (C4), T5 introduced a compute-efficient *span corruption* objective. Rather than masking individual isolated tokens, contiguous spans of text (averaging 3 tokens in length, totaling 15% of the sequence) are replaced by unique sentinel tokens $\langle \text{extra_id_0} \rangle, \langle \text{extra_id_1} \rangle, \ldots$. The target sequence $\mathbf{y}$ consists exclusively of these sentinel tokens followed by their excised original text. By omitting uncorrupted tokens from the decoder targets, T5 drastically reduces the decoder sequence length during pre-training, yielding substantial computational savings over standard sequence reconstruction.

1.1.3 Trade-offs and computational bottlenecks of dual-stack models

Despite the elegance of the text-to-text abstraction, full encoder-decoder architectures impose structural and computational overheads that ultimately motivated researchers to explore single-stack variants.

1. **Parameter Redundancy in Separate Stacks**: An encoder-decoder network maintains two complete Transformer stacks. For an L-layer architecture with hidden dimension d_{model} and feed-forward dimension d_{ff}, the model instantiates L bidirectional encoder blocks and L causal decoder blocks. Furthermore, every decoder layer must maintain

an additional multi-head cross-attention mechanism with parameter projection matrices $\mathbf{W}^{Q}_{\text{cross}}, \mathbf{W}^{K}_{\text{cross}}, \mathbf{W}^{V}_{\text{cross}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ and $\mathbf{W}^{O}_{\text{cross}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. When evaluated against a single-stack architecture of equivalent total parameter volume, parameter capacity is split between encoding and decoding stages.

2. **Cross-Attention Memory and Latency Footprint**: In the decoding phase, generating each output token requires computing attention across the entire encoded representation. For an input sequence of length S and an output sequence of length T, cross-attention requires materializing and storing key-value tensors $\mathbf{K}_{\text{cross}}, \mathbf{V}_{\text{cross}} \in \mathbb{R}^{S \times d_k}$ across all heads and layers. The cumulative cross-attention computational complexity scales as $O(L \cdot S \cdot T \cdot d_{\text{model}})$. In latency-critical deployment scenarios, repeatedly attending over static encoder states creates high memory bandwidth demands.

3. **Asymmetric Computational Allocation**: Natural language tasks exhibit marked structural asymmetry. In pure comprehension problems (e.g., text categorization, document retrieval, grammatical acceptability), generating an autoregressive sequence of tokens via a decoder stack is strictly superfluous; a direct pooling of the encoded states provides sufficient representation. Conversely, in open-ended generative tasks (e.g., creative writing, dialogue, code synthesis), there is no distinct "source" sequence to encode; the prompt is merely the initial conditioning segment of a single unified sequence.

These structural trade-offs led to an architectural fork: removing the autoregressive decoder produced bidirectional *encoder-only* representations, whereas removing the encoder and cross-attention produced causal *decoder-only* generative architectures.

1.2 Encoder-Only Models

When an application demands deep semantic understanding, entity tagging, syntactic parsing, or document classification rather than open-ended text synthesis, the autoregressive decoder constitutes unnecessary computational overhead. By excising the decoder stack entirely and eliminating the causal triangular attention mask, the model gains unconstrained bidirectional context: every token at every layer attends simultaneously to all prior, concurrent, and subsequent tokens.

The archetypal implementation of this paradigm is BERT (*Bidirectional Encoder Representations from Transformers*), introduced by Devlin et al. [**?**]. Figure 1.2 depicts the architectural topology of an encoder-only model.

1.2.1 Input representation and special tokens

BERT constructs input representations by summing three distinct embeddings for each sequence position: token embeddings, segment embeddings, and positional embeddings. Figure 1.3 formalizes this composition.

Formally, for position i in a packed sequence, the input embedding vector $\mathbf{E}_{\text{in}}(i) \in \mathbb{R}^{d_{\text{model}}}$ is defined by:

$$\mathbf{E}_{\text{in}}(i) = \mathbf{E}_{\text{token}}(t_i) + \mathbf{E}_{\text{segment}}(s_i) + \mathbf{E}_{\text{pos}}(i), \tag{1.5}$$

where:

1. $\mathbf{E}_{\text{token}}(t_i) \in \mathbb{R}^{d_{\text{model}}}$ maps subword tokens t_i derived via WordPiece tokenization ($|\mathcal{V}| \approx 30,522$) into continuous embeddings.

2. $\mathbf{E}_{\text{segment}}(s_i) \in \{\mathbf{E}_A, \mathbf{E}_B\}$ denotes a learned segment embedding assigned to distinguish the first sentence (Sentence A) from the contiguous or paired second sentence (Sentence B).

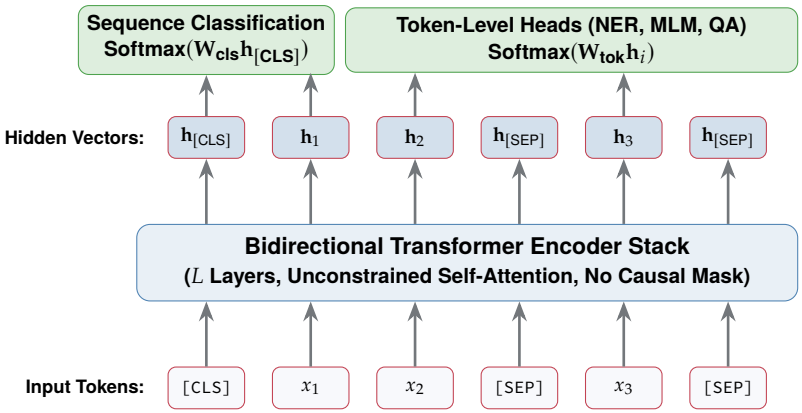


Figure 1.2: The Encoder-only architecture (BERT) [**?**]. By removing the decoder, bidirectional self-attention enables every token representation to condition simultaneously on the entire context. Sequence-level tasks extract the aggregate vector $\mathbf{h}_{[\text{CLS}]}$, while token-level tagging and span tasks branch directly from individual hidden vectors $\mathbf{h}_i$.

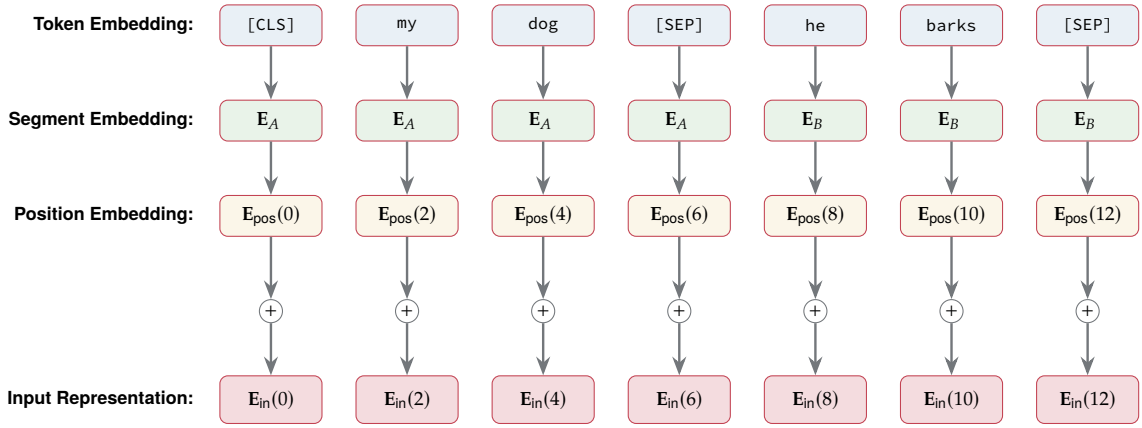


Figure 1.3: BERT input representation structure [**?**]. The final input representation $\mathbf{E}_{\text{in}}(i)$ is the element-wise sum of the subword token embedding $\mathbf{E}_{\text{token}}$, the binary sentence segment embedding $\mathbf{E}_{\text{segment}} \in \{\mathbf{E}_A, \mathbf{E}_B\}$, and the learned absolute positional embedding $\mathbf{E}_{\text{pos}}(i)$.

3. $\mathbf{E}_{\text{pos}}(i) \in \mathbb{R}^{d_{\text{model}}}$ represents a learned absolute positional embedding index up to a context length of 512 tokens.

To structure multi-sentence reasoning and facilitate task classification, BERT introduces two essential delimiter tokens:

- `[CLS]` (*Classification*): Always prepended as the zeroth token (t_0) of every sequence. Because self-attention is unconstrained, the hidden state $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{d_{\text{model}}}$ accumulates contextual information across all tokens, functioning as an aggregate sequence representation for classification heads.
- `[SEP]` (*Separator*): Inserted between Sentence A and Sentence B, and appended at the final position of the sequence. It acts as an unambiguous structural barrier between distinct semantic segments.

1.2.2 Pre-training task 1: Masked Language Model (MLM)

In an unconstrained bidirectional encoder, standard autoregressive next-token prediction is mathematically degenerate: since token i directly attends to token $i+1$, predicting x_{i+1} would

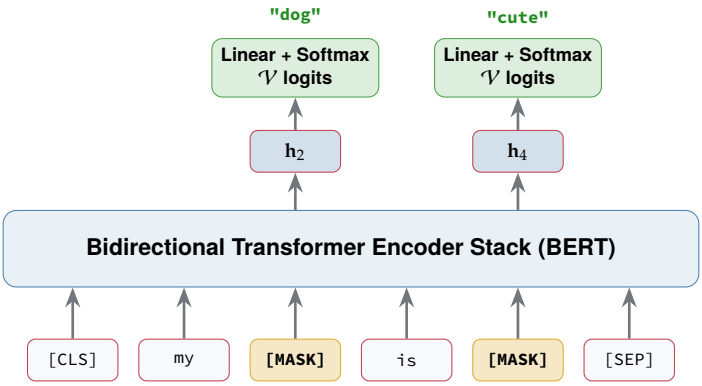


Figure 1.4: The Masked Language Model (MLM) pre-training task in BERT. Approximately 15% of input tokens are corrupted. The final contextual hidden states corresponding to corrupted indices are projected onto the vocabulary vocabulary space through a shared linear layer to reconstruct the original tokens.

trivially reduce to reading the unmasked future token. To train a deep bidirectional representation without informational leakage, Devlin et al. [**?**] designed the *Masked Language Model* (MLM) objective.

Figure 1.4 shows the MLM execution flow. In each training sequence, approximately 15% of all WordPiece tokens are selected uniformly at random for prediction. However, naively replacing every chosen token with a fixed `[MASK]` token introduces a severe *pre-training/fine-tuning discrepancy*: the `[MASK]` token never appears during downstream fine-tuning.

To mitigate this distribution shift, Devlin et al. introduced an 80/10/10 corruption schedule for the selected 15% candidate positions:

1. 80% **of the time**: The token is replaced with the special symbol `[MASK]`.
2. 10% **of the time**: The token is replaced with a randomly sampled token from the vocabulary $\mathcal{V}$. This induces contextual vigilance: because any token might be an injected error, the network cannot simply learn that non-`[MASK]` tokens are always correct.
3. 10% **of the time**: The token is left unaltered. This forces the encoder to maintain a faithful representation of the true observed token.

Formally, let $\mathcal{M}$ denote the set of corrupted token indices. For each index $i \in \mathcal{M}$, the final contextual hidden vector $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$ is passed through a shared linear projection $\mathbf{W}_{\text{mlm}} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ followed by a softmax:

$$P(x_i = v \mid \mathbf{h}_i) = \frac{\exp(\mathbf{w}_v^\top \mathbf{h}_i + b_v)}{\sum_{v' \in \mathcal{V}} \exp(\mathbf{w}_{v'}^\top \mathbf{h}_i + b_{v'})}. \tag{1.6}$$

The MLM loss is computed strictly over the corrupted positions:

$$\mathcal{L}_{\text{MLM}}(\boldsymbol{\theta}) = -\sum_{i \in \mathcal{M}} \log P(x_i = t_i^* \mid \mathbf{h}_i; \boldsymbol{\theta}), \tag{1.7}$$

where t_i^* is the ground-truth token identity before corruption.

1.2.3 Pre-training task 2: Next Sentence Prediction (NSP)

While MLM develops fine-grained token-level contextual representations, many downstream NLP applications (such as Natural Language Inference and Question Answering) require

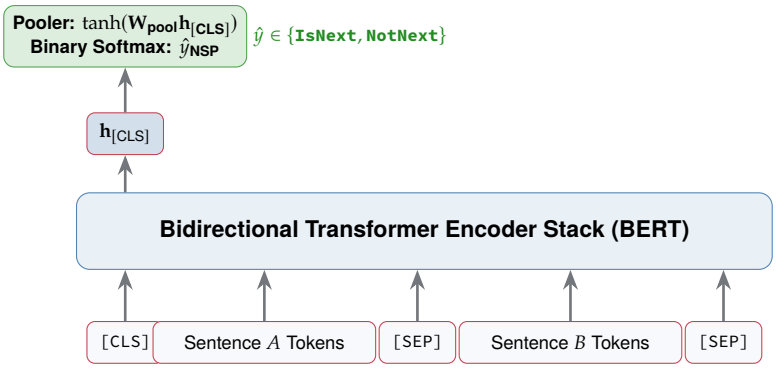


Figure 1.5: The Next Sentence Prediction (NSP) pre-training task in BERT. Two sentences are packed into a single input separated by `[SEP]`. The final pooled representation of `[CLS]` is classified to determine whether Sentence B logically follows Sentence A.

modeling relational semantics between pairs of sentences. To explicitly train sentence relationship modeling, BERT incorporates the *Next Sentence Prediction* (NSP) binary classification objective.

Figure 1.5 illustrates the NSP formulation. Input sequences are assembled by concatenating sentence pairs:

$$\mathbf{x} = (\texttt{[CLS]}, a_1, \ldots, a_{|A|}, \texttt{[SEP]}, b_1, \ldots, b_{|B|}, \texttt{[SEP]}). \tag{1.8}$$

Training instances are sampled according to a balanced binary protocol:

- 50% **of instances (Positive Class: `IsNext`)**: Sentence B is the actual contiguous sentence following Sentence A in the source corpus.
- 50% **of instances (Negative Class: `NotNext`)**: Sentence B is a random sentence sampled independently from another document in the corpus.

At the model output, the hidden state corresponding to the first token, $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{d_{\text{model}}}$, is mapped through a non-linear pooler layer with parameter matrix $\mathbf{W}_{\text{pool}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$:

$$\mathbf{c} = \tanh(\mathbf{W}_{\text{pool}} \mathbf{h}_{[\text{CLS}]} + \mathbf{b}_{\text{pool}}). \tag{1.9}$$

A binary classification projection $\mathbf{W}_{\text{NSP}} \in \mathbb{R}^{2 \times d_{\text{model}}}$ computes the class probabilities:

$$P(\text{IsNext} \mid \mathbf{c}) = \frac{\exp(\mathbf{w}_{\text{next}}^{\top} \mathbf{c})}{\exp(\mathbf{w}_{\text{next}}^{\top} \mathbf{c}) + \exp(\mathbf{w}_{\text{not}}^{\top} \mathbf{c})}. \tag{1.10}$$

The cumulative pre-training loss combines the MLM and NSP objectives over every mini-batch:

$$\mathcal{L}_{\text{total}}(\boldsymbol{\theta}) = \mathcal{L}_{\text{MLM}}(\boldsymbol{\theta}) + \mathcal{L}_{\text{NSP}}(\boldsymbol{\theta}). \tag{1.11}$$

1.2.4 Attention map anatomy and emergent linguistic structures

An instructive insight from the BERT architecture is that multi-head bidirectional self-attention naturally develops specialized linguistic capabilities without explicit syntactic supervision. In $\text{BERT}_{\text{BASE}}$, the model instantiates $L = 12$ layers with $H = 12$ attention heads per layer, computing:

$$N_{\text{maps}} = 12 \times 12 = 144 \tag{1.12}$$

distinct attention matrices per sequence. For a sequence of length N, each attention map is an $N \times N$ matrix $\mathbf{A} \in [0, 1]^{N \times N}$ where entry $A_{i,j}$ indicates the attention weight that token i assigns to token j.

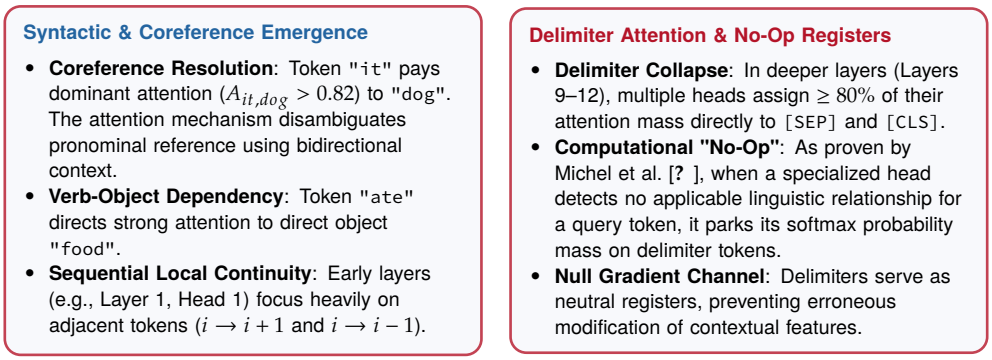


Figure 1.6: Emergent linguistic specialization and delimiter behavior across BERT attention heads [? ?]. Middle layers resolve syntactic dependencies and coreferences, while deeper layers utilize delimiter tokens as computational no-op registers.

In course lectures, the internal mechanics of these 144 attention heads are analyzed using the running sentence:

"[CLS] The dog ate the food because it was hungry [SEP]"

comprising $N = 11$ tokens. Empirical probing of these attention maps, formalizing studies by Clark et al. [?] and Michel et al. [?], reveals two striking emergent phenomena:

1. **Syntactic and Coreference Specialization**: In middle layers (Layers 5 through 8), individual attention heads function as soft syntactic parsers. For example, when resolving the pronoun "it", specific heads route their primary attention mass ($A_{i,j} > 0.80$) back to the antecedent noun phrase "dog". Because the self-attention formula is unconstrained, token representations dynamically absorb semantic properties from their syntactic referents. Similarly, verbs attend directly to their objects (e.g., "ate" attends strongly to "food").

2. **Delimiter Registers as Computational "No-Ops"**: In higher layers (Layers 9 through 12), an intriguing behavior emerges: an overwhelming fraction of query tokens assign the vast majority of their attention probability to delimiter tokens, predominantly [SEP] and [CLS]. Clark et al. [?] established that delimiter tokens act as *computational no-op registers*. Softmax requires rows of the attention matrix to sum strictly to 1:

$$\sum_{j=1}^{N} A_{i,j} = 1. \tag{1.13}$$

   If a specialized head (e.g., a preposition detector) evaluates a query token that has no relevant dependency in the sentence, it cannot emit a zero vector from softmax. To prevent injecting spurious semantic information into the residual stream, the head routes its attention mass to [SEP], whose representation functions as a neutral null register.

1.2.5 Downstream task adaptation

To adapt the pre-trained encoder to specific downstream tasks, the pre-training MLM and NSP projection layers are discarded and replaced with lightweight, task-specific output layers. The entire network is then fine-tuned end-to-end:

1. **Sentence-Level Classification (e.g., SST-2, MNLI)**: The classification head maps the aggregate representation $\mathbf{h}_{[\text{CLS}]}$ through a projection matrix $\mathbf{W}_{\text{cls}} \in \mathbb{R}^{K \times d_{\text{model}}}$:

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{W}_{\text{cls}}\mathbf{h}_{[\text{CLS}]} + \mathbf{b}_{\text{cls}}). \tag{1.14}$$

2. **Token-Level Tagging (e.g., NER, POS Tagging)**: Each token position i passes its hidden vector $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$ to a shared linear projection $\mathbf{W}_{\text{tok}} \in \mathbb{R}^{C \times d_{\text{model}}}$:

$$\hat{\mathbf{y}}_i = \text{Softmax}(\mathbf{W}_{\text{tok}}\mathbf{h}_i + \mathbf{b}_{\text{tok}}). \tag{1.15}$$

3. **Extractive Question Answering (e.g., SQuAD)**: The model learns two parameter vectors, $\mathbf{s}, \mathbf{e} \in \mathbb{R}^{d_{\text{model}}}$, corresponding to span start and span end predictors. The probability that token i marks the start of the answer span is given by:

$$P_{\text{start}}(i) = \frac{\exp(\mathbf{s}^\top \mathbf{h}_i)}{\sum_{j=1}^{N} \exp(\mathbf{s}^\top \mathbf{h}_j)}, \tag{1.16}$$

with an identical formulation parameterized by $\mathbf{e}$ for the span terminus.

1.3 Decoder-Only Models

While bidirectional encoders excel at discriminative understanding and sequence labeling, they cannot naturally synthesize open-ended natural text. Generating a sequence autoregressively with an unconstrained bidirectional model requires repeatedly re-encoding candidate completions with iterative masking heuristics, which is computationally inefficient.

The causal *decoder-only* architecture resolves this limitation by discarding the encoder stack and cross-attention blocks entirely. The prompt sequence is supplied directly as the initial prefix of the decoder stack, and generation proceeds sequentially via strictly causal self-attention.

1.3.1 Causal autoregressive formulation and architecture topology

In a decoder-only architecture, the joint probability of an arbitrary sequence $\mathbf{x} = (x_1, x_2, \ldots, x_N)$ over vocabulary $\mathcal{V}$ is decomposed via the exact probability chain rule without Markov independence assumptions:

$$P(\mathbf{x}) = \prod_{i=1}^{N} P(x_i \mid x_1, x_2, \ldots, x_{i-1}) = \prod_{i=1}^{N} P(x_i \mid \mathbf{x}_{<i}). \tag{1.17}$$

Taking the negative logarithm yields the canonical autoregressive next-token prediction objective:

$$\mathcal{L}_{\text{AR}}(\boldsymbol{\theta}) = -\sum_{i=1}^{N} \log P(x_i \mid \mathbf{x}_{<i}; \boldsymbol{\theta}). \tag{1.18}$$

To compute representations for all positions $i \in \{1, \ldots, N\}$ in parallel during training without violating the causal factorization in Equation (1.17), the architecture enforces strict temporal masking within every self-attention layer. Let $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{N \times d_k}$ denote the query and key projection matrices for a given attention head. The pre-softmax logit matrix is augmented by an additive causal mask $\mathbf{M} \in \{0, -\infty\}^{N \times N}$:

$$\mathbf{A} = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right), \tag{1.19}$$

where the mask matrix $\mathbf{M}$ is defined by:

$$M_{i,j} = \begin{cases} 0 & \text{if } j \le i, \\ -\infty & \text{if } j > i. \end{cases} \tag{1.20}$$

Because $\exp(-\infty) = 0$, attention weights for all future positions $j > i$ are zero. Thus, the contextual hidden state $\mathbf{h}_i \in \mathbb{R}^{d_{\text{model}}}$ at position i is an exact deterministic function of the prefix $\mathbf{x}_{\le i}$ and remains independent of subsequent tokens $x_{j>i}$.

1.3.2 Architectural streamlining: eliminating encoder and cross-attention

Relative to the canonical Transformer, the decoder-only architecture simplifies the processing pipeline:

1. **Elimination of the Encoder Stack**: Rather than projecting source text into an intermediate representation space via an independent encoder, the conditioning prompt is simply ingested as the initial prefix of the decoder sequence.
2. **Excise of Cross-Attention Sub-Layers**: Because there is no external encoder state, each decoder block contains only two core components: a causal masked multi-head self-attention layer and a position-wise feed-forward network. This reduces the number of matrix projections per layer from eight ($\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$ for self-attention; $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$ for cross-attention) to four.
3. **Training Parallelism via Teacher Forcing**: Although inference is necessarily sequential (generating one token at a time), training remains fully parallel. The ground-truth sequence is supplied as input, and causal triangular masking ensures that predictions for all positions are computed in a single forward pass under the teacher-forcing paradigm.

1.3.3 Convergence on the decoder-only paradigm

The NLP research community progressively standardized on decoder-only architectures for large-scale foundation models. This convergence was driven by fundamental computational and operational advantages:

1. **Homogeneous Key-Value Cache Memory Layout**: In an encoder-decoder architecture, autoregressive generation requires maintaining two separate key-value caches: a static encoder cache $\mathbf{K}_{\text{enc}}, \mathbf{V}_{\text{enc}} \in \mathbb{R}^{S \times d_k}$ and a growing decoder cache $\mathbf{K}_{\text{dec}}, \mathbf{V}_{\text{dec}} \in \mathbb{R}^{t \times d_k}$. Managing heterogeneous memory buffers across distributed accelerator clusters complicates tensor-parallel and pipeline-parallel execution. In contrast, decoder-only architectures maintain a single, append-only key-value cache buffer.
2. **Universal In-Context Sequence Interface**: By unifying input context and output completions into a single autoregressive sequence, decoder-only models natively support few-shot prompting, multi-turn dialogue, and chain-of-thought reasoning without requiring specialized architectural modifications or task-specific fine-tuning.

The empirical scaling laws, pre-training corpus curation strategies, and historical milestones (from GPT-1 and GPT-2 to massive modern parameter regimes) that established this paradigm's dominance are examined comprehensively in Chapter **??**.

1.4 Autoregressive Sampling Strategies

Regardless of whether an architecture employs an encoder-decoder or decoder-only topology, generating text requires converting continuous output vectors into discrete token sequences. At generation step t, the final hidden state $\mathbf{h}_t \in \mathbb{R}^{d_{\text{model}}}$ is projected onto vocabulary $\mathcal{V}$ via the unembedding matrix $\mathbf{W}_U \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ to yield raw logits $\mathbf{z}_t \in \mathbb{R}^{|\mathcal{V}|}$:

$$\mathbf{z}_t = \mathbf{W}_U \mathbf{h}_t + \mathbf{b}_U. \tag{1.21}$$

Applying the softmax operator produces a categorical probability distribution over all known tokens:

$$P(x_t = v \mid \mathbf{x}_{<t}) = \frac{\exp(z_{t,v})}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_{t,j})}, \quad \forall v \in \{1, \ldots, |\mathcal{V}|\}. \tag{1.22}$$

Selecting discrete tokens $\hat{x}_t$ from this distribution is governed by the *decoding strategy*. As established in course lectures, selecting tokens naively via probability maximization leads to degenerative failure modes. This section formalizes the complete taxonomy of deterministic and stochastic decoding algorithms, illustrated through the running course generation benchmark based on the Sherlock Holmes prompt from Slides 98–112.

1.4.1 The decoding dilemma: maximization versus degeneration

A natural formulation of text generation is maximum a posteriori (MAP) decoding, seeking the sequence $\mathbf{y}^* = (\hat{x}_1, \ldots, \hat{x}_T)$ that maximizes the joint conditional likelihood:

$$\mathbf{y}^* = \arg\max_{\mathbf{y}} P(\mathbf{y} \mid \mathbf{x}_{\text{prompt}}) = \arg\max_{\mathbf{y}} \sum_{t=1}^{T} \log P(x_t \mid \mathbf{x}_{<t}). \tag{1.23}$$

Exact maximization over the combinatorial space $\mathcal{V}^T$ requires evaluating $|\mathcal{V}|^T$ trajectories, which is computationally intractable for modern vocabularies ($|\mathcal{V}| \geq 32,000$, $T \geq 100$). However, even when approximated, pure likelihood maximization exhibits severe failure modes in open-ended natural text generation. As demonstrated by Holtzman et al. [**?**], human language does not maximize probability at every step. Maximization decoding frequently causes *degeneration*, where the model becomes trapped in repetitive loops or emits generic, uninformative phrases.

1.4.2 Deterministic decoding: greedy search and beam search

Deterministic decoding strategies map a logit distribution to a unique, reproducible output sequence without stochastic sampling.

Greedy Decoding Greedy search selects the single token with the highest predicted probability at each step:

$$\hat{x}_t = \arg\max_{v \in \mathcal{V}} P(x_t = v \mid \mathbf{x}_{<t}) = \arg\max_{v \in \mathcal{V}} z_{t,v}. \tag{1.24}$$

While computationally minimal ($O(1)$ selection cost), greedy decoding suffers from two fatal deficiencies:

- **Search Myopia**: Greedy decoding cannot look beyond the immediate token. Selecting an apparently optimal token at step t can steer the network into a low-probability manifold at step $t+1$, missing globally higher-likelihood sequences that began with a lower-probability initial token.
- **Cyclic Repetition Loops**: In open-ended generation, greedy selection reliably causes infinite repetitive loops.

This pathology is highlighted in Slide 100 of the course materials. When conditioning GPT-2 on the Arthur Conan Doyle prompt:

> *"I had called upon my friend, Mr. Sherlock Holmes, one day in the autumn of last year and found him in deep conversation with a very stout, florid-faced, elderly gentleman with fiery red hair. With an apology for my intrusion, I was about to withdraw when Holmes pulled me abruptly into the room and closed the door behind me."*

greedy decoding produces the following degenerate continuation:

```
"I am sorry, Mr.  Holmes," I said, "but I am not going to tell you what
I have seen.  I am afraid I have not seen you in a long time.  I am afraid
I have not seen you in a long time.  I am afraid I have not seen you in
a long time.  I am afraid I have not seen you in a long time.  I am afraid
I have not seen..."
```

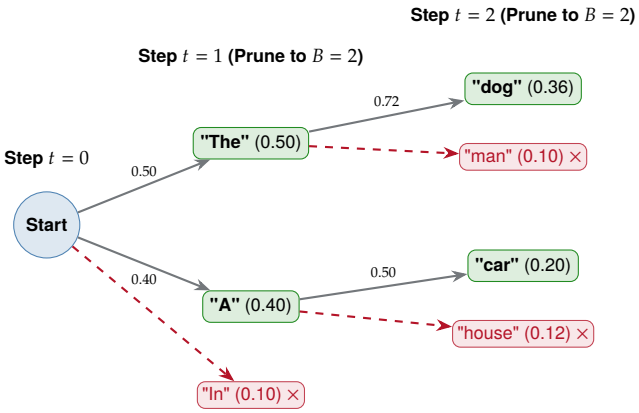


Figure 1.7: Beam search decoding with beam width $B = 2$ (inspired by Slide 101). At each step, all B active sequences are expanded across vocabulary candidates and pruned back to the B sequences with highest cumulative probability.

Once a token sequence repeats, the self-attention mechanism reinforces the historical pattern, driving the model into an inescapable degenerative attractor. To mitigate this loop, generation frameworks often introduce *repetition penalties*:

$$z'_{t,v} = \begin{cases} z_{t,v}/\theta & \text{if } z_{t,v} > 0 \text{ and } v \in \mathbf{x}_{<t}, \\ z_{t,v} \cdot \theta & \text{if } z_{t,v} \leq 0 \text{ and } v \in \mathbf{x}_{<t}, \end{cases} \tag{1.25}$$

where $\theta > 1.0$ penalizes previously generated tokens.

Beam Search Beam search expands the greedy horizon by tracking the top B most probable partial sequences (the *beam width*) at each step. Let $\mathcal{B}_{t-1} = \{\mathbf{y}_1, \ldots, \mathbf{y}_B\}$ denote the set of active hypotheses at step $t-1$. At step t, the algorithm evaluates the Cartesian product of all active beams and the entire vocabulary:

$$C_t = \mathcal{B}_{t-1} \times \mathcal{V} = \{(\mathbf{y}_b, v) \mid \mathbf{y}_b \in \mathcal{B}_{t-1}, v \in \mathcal{V}\}, \tag{1.26}$$

scoring each candidate trajectory by its cumulative log-likelihood:

$$\text{Score}(\mathbf{y}_b \circ v) = \log P(\mathbf{y}_b \mid \mathbf{x}) + \log P(v \mid \mathbf{x}, \mathbf{y}_b). \tag{1.27}$$

The set is then pruned back to the top B candidates:

$$\mathcal{B}_t = \arg \max_{\mathcal{B} \subset C_t, |\mathcal{B}|=B} \sum_{\mathbf{y} \in \mathcal{B}} \text{Score}(\mathbf{y}). \tag{1.28}$$

Because cumulative probability monotonically decreases with sequence length ($\log P \leq 0$), raw beam search exhibits a strong structural bias toward prematurely terminating with short sequences. To counter this, practitioners apply a length normalization penalty:

$$\mathcal{S}(\mathbf{y}) = \frac{\sum_{t=1}^{T} \log P(y_t \mid \mathbf{y}_{<t}, \mathbf{x})}{(5+T)^\alpha / (5+1)^\alpha}, \tag{1.29}$$

where $\alpha \in [0.6, 1.0]$ scales length normalization.

While beam search is standard for deterministic, constrained sequence mapping (e.g., machine translation and abstractive summarization), it is poorly suited for open-ended generation: high probability beams converge on repetitive, bland text.

1.4.3 Stochastic sampling: temperature scaling, Top-k, and Top-p

To generate natural, non-repetitive text, decoding must draw tokens stochastically from the predicted distribution: $\hat{x}_t \sim P(x_t \mid \mathbf{x}_{<t})$. However, unrestricted sampling introduces vulnerabilities that require calibrated truncation.

Pure Random Sampling and the Long Tail Risk Under pure multinomial sampling, the next token is sampled directly from the raw softmax distribution over all $|\mathcal{V}|$ tokens. While this guarantees non-deterministic outputs, it exposes generation to the *long tail risk*.

Modern vocabularies contain tens of thousands of tokens ($|\mathcal{V}| \geq 32,000$). Even when individual probability values for implausible words are small (e.g., $P \approx 10^{-4}$), their cumulative probability mass across thousands of tail tokens is substantial. Inevitably, the random draw samples an incoherent or contextually inappropriate token from this distribution tail, permanently derailing subsequent generation.

This failure mode is shown in Slide 103. Under pure random sampling on the Sherlock Holmes prompt, the continuation rapidly devolves into syntactic disarray:

```
"...It was alarmed me, as I had never heard of him before but last, yet
I went to put something into his hand which I carried open and which I
then pocketed into the drawer, so as not to be seen until soon afterwards.
Then it was as though in a dream, and my amazement owled wild-it seems
an impossibility..."
```

Temperature Scaling Temperature scaling modulates the entropy of the categorical distribution by dividing raw logits by a positive hyperparameter $T > 0$ prior to the softmax operator:

$$P_T(x_t = v \mid \mathbf{x}_{<t}) = \frac{\exp(z_{t,v}/T)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_{t,j}/T)}. \tag{1.30}$$

The mathematical behavior across different temperature regimes is bounded by two analytical limits:

- **Limit $T \to 0^+$ (Argmax / Zero Entropy)**: Dividing logits by an infinitesimal value exaggerates differences between logits, collapsing the distribution to a Dirac delta centered at the mode:
$$\lim_{T \to 0^+} P_T(x_t = v \mid \mathbf{x}_{<t}) = \begin{cases} 1 & \text{if } v = \arg\max_j z_{t,j}, \\ 0 & \text{otherwise,} \end{cases} \tag{1.31}$$
recovering deterministic greedy search.
- **Limit $T \to \infty$ (Uniform Distribution / Maximum Entropy)**: As T approaches infinity, scaled logits converge to zero ($z_{t,v}/T \to 0$), flattening softmax into a discrete uniform distribution:
$$\lim_{T \to \infty} P_T(x_t = v \mid \mathbf{x}_{<t}) = \frac{1}{|\mathcal{V}|}, \quad \forall v \in \mathcal{V}. \tag{1.32}$$

The qualitative impact of temperature scaling is captured in Slides 109–112 using the Sherlock Holmes benchmark:

- **Low Temperature ($T = 0.1$, Slide 109)**: Near-complete probability collapse into repetitive loops:

```
"...I am sorry, Mr.  Holmes, for having been so rude to you.  I am
sorry, Mr.  Holmes, for having been so rude to you.  I am sorry, Mr.
Holmes, for having been so rude to you..."
```

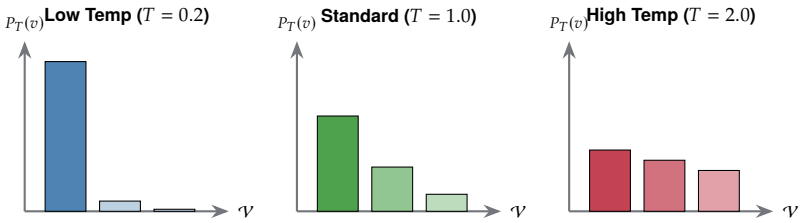


Figure 1.8: Softmax distribution modulation across temperature regimes. Low temperatures ($T < 1.0$) sharpen probability mass toward the mode, while high temperatures ($T > 1.0$) flatten the distribution toward uniform entropy.

- **Balanced Low Temperature ($T = 0.5$, Slide 110)**: Strong coherence, consistent tone, and natural progression:

  ```
  "...It was a very sad moment, but a true one.  After a short time, I
  was ushered out of the room by the old gentleman, who had come to ask
  me to come with him to the house.  I was quite at ease, and when he
  told me that he was going to look after me, I said, 'This is not the
  first time I've seen you.'..."
  ```

- **High Temperature ($T = 1.1$, Slide 111)**: Increasing lexical diversity with emerging contextual drift:

  ```
  "...We remained there four or five minutes, staring at every hem from
  the last person who might show up in my passageway.  It was dark—black—but
  dark, almost this glintry and slant field..."
  ```

- **Very High Temperature ($T = 1.5$, Slide 112)**: Syntactic collapse into hallucinatory gibberish:

  ```
  "...He kept Columbus visibly busy trying secretly rendered strawberries
  with a reel motif.  This anecdote barely IMLEG it off it off 138 ALK###
  elsewhere where 4 doctor m Hiragh benecameth [aintedie$ andalaptag..."
  ```

Top-k Sampling (Fixed Cardinality Truncation) Introduced by Fan et al. [**?**], Top-k sampling addresses the long-tail vulnerability by restricting the sampling pool to the k most probable tokens. Formally, tokens are sorted in descending order of probability:

$$P(x^{(1)} \mid \mathbf{x}_{<t}) \geq P(x^{(2)} \mid \mathbf{x}_{<t}) \geq \cdots \geq P(x^{(|\mathcal{V}|)} \mid \mathbf{x}_{<t}). \tag{1.33}$$

The sampling support is truncated to the subset $V^{(k)} = \{x^{(1)}, \ldots, x^{(k)}\} \subset \mathcal{V}$, and probabilities are renormalized:

$$P_{\text{top-}k}(x_t = v \mid \mathbf{x}_{<t}) = \begin{cases} \frac{P(x_t=v|\mathbf{x}_{<t})}{\sum_{j \in V^{(k)}} P(x_t=j|\mathbf{x}_{<t})} & \text{if } v \in V^{(k)}, \\ 0 & \text{otherwise.} \end{cases} \tag{1.34}$$

Under Top-k sampling with $k = 50$ (Slide 105), the Sherlock Holmes continuation maintains syntactic structure and thematic tone without degenerative loops:

```
"...'You will think, Doctor, that a man of my age would be so much of a
menace to me, if I were to refuse a pass for him.  I should rather get
a few things away from him than any which might injure his reputation...'"
```

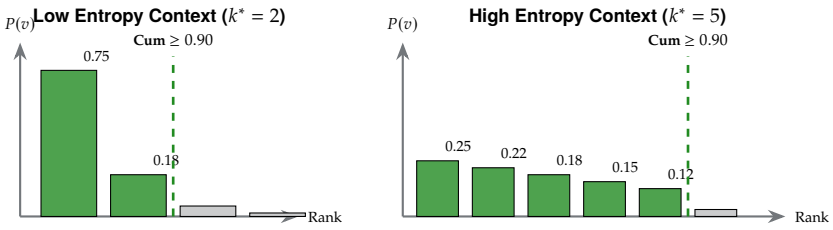


Figure 1.9: Top-p (Nucleus) sampling mechanism. The candidate pool dynamically expands and contracts based on model confidence: in low-entropy contexts only 2 tokens meet the $p = 0.90$ mass threshold, whereas in high-entropy contexts the pool expands to 5 tokens.

However, Top-k suffers from a core structural flaw: *fixed cardinality cannot adapt to contextual uncertainty*. In low-entropy contexts where the model is highly confident (e.g., `"The Eiffel Tower is in"` $\rightarrow$ `"Paris"` with $P = 0.98$), a fixed $k = 50$ forces the algorithm to sample among 49 implausible tokens. Conversely, in high-entropy contexts where hundreds of valid completions exist, $k = 50$ truncates plausible continuations.

Top-p / Nucleus Sampling (Adaptive Cumulative Mass Truncation) To eliminate the rigidity of fixed cardinality, Holtzman et al. [**?**] introduced *Nucleus* (Top-p) sampling. Instead of clipping at a fixed rank k, nucleus sampling defines an adaptive candidate set $V^{(p)} \subset \mathcal{V}$ containing the minimal set of tokens whose cumulative probability mass meets or exceeds a target threshold $p \in (0, 1]$:

$$\sum_{v \in V^{(p)}} P(x_t = v \mid \mathbf{x}_{<t}) \geq p. \tag{1.35}$$

Formally, letting tokens be sorted in descending probability order, the cutoff index k^* is determined by:

$$k^* = \min \left\{ K \in \{1, \ldots, |\mathcal{V}|\} : \sum_{i=1}^{K} P(x^{(i)} \mid \mathbf{x}_{<t}) \geq p \right\}. \tag{1.36}$$

The candidate pool is defined by $V^{(p)} = \{x^{(1)}, \ldots, x^{(k^*)}\}$, and probabilities are renormalized over this dynamic set:

$$P_{\text{top-}p}(x_t = v \mid \mathbf{x}_{<t}) = \begin{cases} \frac{P(x_t = v \mid \mathbf{x}_{<t})}{\sum_{j \in V^{(p)}} P(x_t = j \mid \mathbf{x}_{<t})} & \text{if } v \in V^{(p)}, \\ 0 & \text{otherwise.} \end{cases} \tag{1.37}$$

As illustrated in Figure 1.9, nucleus sampling adapts to contextual entropy:

- When the model is confident, the top few tokens capture most probability mass; the candidate set collapses to $k^* \in \{1, 2, 3\}$, eliminating spurious tail tokens.
- When the distribution is diffuse, the candidate pool automatically expands to hundreds of plausible tokens, preserving creative diversity.

Under nucleus sampling ($p = 0.9$, Slide 107), generation maintains high fluency, coherent narrative progression, and natural lexical variation:

```
"...Mr.  Holmes had been talking for a while and seemed very engrossed
in the conversation.  He wished to conduct his last spoke through the eyes
of another, and by that he meant to tell me the whole of the publick story
that had been passed from one of them into the other..."
```

Table 1.1: Comparative analysis of autoregressive decoding strategies.

Decoding Strategy	Type	Complexity	Primary Failure Mode	Optimal Task Regime
Greedy Search	Deterministic	$O(1)$	Search myopia, cyclic loops	Strict mathematical/code tasks
Beam Search	Deterministic	$O(B \cdot \lvert\mathcal{V}\rvert)$	Repetitive generic phrasing	Translation, summarization
Random Sampling	Stochastic	$O(\lvert\mathcal{V}\rvert)$	Severe long-tail derailment	High-entropy creative text
Temperature	Stochastic	$O(\lvert\mathcal{V}\rvert)$	Extreme entropy drift ($T > 1.5$)	General text generation
Top-k Sampling	Truncated Stoch.	$O(\lvert\mathcal{V}\rvert \log k)$	Inflexible fixed pool size	Stylized controlled generation
Top-p (Nucleus)	Truncated Stoch.	$O(\lvert\mathcal{V}\rvert \log \lvert\mathcal{V}\rvert)$	Occludes rare valid idioms	Open-ended dialogue, chat

1.4.4 Empirical decoding comparison and selection guide

Table 1.1 synthesizes the mathematical properties, complexity profiles, and operational trade-offs of all six decoding strategies.

In production language modeling systems, practitioners typically chain these mechanisms into a unified multi-stage decoding pipeline:

$$\mathbf{z} \xrightarrow{\text{Repetition Penalty}} \mathbf{z}' \xrightarrow{\text{Temperature } T} \mathbf{z}'' \xrightarrow{\text{Top-}k \text{ Truncation}} \mathbf{p}' \xrightarrow{\text{Top-}p \text{ Cutoff}} \hat{x}_t \sim \text{Categorical}(\mathbf{p}''). \tag{1.38}$$

First, repetition penalties adjust logits of previously emitted tokens. Second, temperature scaling calibrates entropy. Third, Top-k filtering eliminates the outer tail. Fourth, Top-p thresholding dynamically isolates the nucleus. Finally, the next token is sampled from the renormalized distribution.